From Simulation to Silicon: Fast and Exact YOLO26 Object Detection on Low-Cost Microcontrollers

Billal Boumedine
Faculty of Computer Science, USTHB
Algiers, Algeria
boumedinebillal@gmail.com

Amina Belkacem
Faculty of Computer Science, USTHB
Algiers, Algeria
aminabelkacem.cv@gmail.com

Naima Boukhiar*
Higher School of Computer Science
Algiers, Algeria
naima.boukhiar@univ-alger.dz

Abstract

Object detection is increasingly expected to run directly on ultra-low-cost microcontrollers rather than in the cloud, yet no deployment of the NMS-free YOLO26 detector had been demonstrated on the $5, GPU-free ESP32-P4, and its depthwise, quantization-sensitive detection head made naive post-training quantization fail outright. We hypothesized that closing this gap requires a multi-level optimization that co-designs the model's computation graph together with the hardware kernels that execute it, rather than optimizing either in isolation. We deployed YOLO26n (2.4M parameters) on the ESP32-P4 (400 MHz RISC-V, a 16×INT8 PIE SIMD coprocessor, 768 KB SRAM, 32 MB PSRAM) using mixed-precision INT16/INT8 quantization validated by a custom bit-exact LUT emulator (`esp_ppq_lut`). We then built a Neural Morphing Engine that applies block-by-block graph transformations—selecting a target operation, splicing in a replacement, distilling it against a frozen teacher on 512 unlabeled images, and gating acceptance on cosine similarity and magnitude ratio—to produce hardware-friendly operators, paired with four custom PIE SIMD kernels (TransposePIE, NN-LUTINT16, HardSiLUPie8, TiledConvBlock). The mixed-precision baseline reached 2,072 ms inference at 0.365 mAP50-95, already 1.33× faster than the official YOLOv11n deployment. Co-designed graph morphing and hardware kernels then reduced inference to 1,088 ms—a further 1.90× (47.5%) speedup—yielding 0.361 mAP50-95 (only −3.7% from the FP32 reference of 0.375) and a 2.54× overall speedup over YOLOv11n at comparable accuracy. Validation showed 100% bit-exact parity between the Python simulation and the on-device RISC-V output (0 of 451,584 output values in error), whereas bypassing the emulator produced divergence in 88% of values. The baseline was merged into Espressif's official esp-dl SDK (PR #286). A bit-exact digital twin and memory-aware hardware kernels close two independent gaps—simulation fidelity and the mismatch between FLOPs-centric design and memory-bound silicon—and neither gap is closed by graph transformation or kernel engineering alone. This co-design pattern offers a template for shipping correct, fast, NMS-free detectors on commodity RISC-V microcontrollers, supporting the deployment of capable perception across fleets of low-cost, distributed IoT devices.

CCS Concepts

• **Computing methodologies** → **Object detection**; • **Hardware** → **Emerging architectures**; • **Networks** → *Sensor networks.*

Keywords

TinyML, edge inference, object detection, YOLO26, RISC-V, microcontrollers, quantization, hardware-software co-design, SIMD kernels, IoT

ACM Reference Format:
Billal Boumedine, Amina Belkacem, and Naima Boukhiar. 2026. From Simulation to Silicon: Fast and Exact YOLO26 Object Detection on Low-Cost Microcontrollers. In *AIoT '26: The 3rd International Workshop on the Integration of Distributed Machine Learning and the Internet of Things (AIoT), November 26, 2026, Tokyo, Japan.* ACM, New York, NY, USA, 6 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

TinyML and edge inference have moved from a niche research interest to the default expectation for on-device intelligence. Empirical benchmarking of deep neural network quantization and deployment pipelines across a range of commercial microcontrollers has shown that INT8 post-training quantization is now a standard, reproducible route to fitting convolutional networks within sub-megabyte flash and SRAM budgets, at latencies compatible with real-time sensing [10]. Independent measurements of a distributed edge-IoT architecture running machine learning inference across low-power ARM nodes further confirm that resource-constrained, non-GPU hardware can match or exceed the estimation accuracy of centralized, GPU-backed pipelines while cutting energy consumption by roughly a third [4]. These results, drawn from real deployed hardware rather than simulation, establish that the open constraint is no longer whether inference can run at the edge, but how much of a modern model's accuracy and latency budget survives the trip from a floating-point checkpoint to fixed-point silicon.

The urgency of closing that gap is sharpened by object detection specifically, where recent empirical deployments quantify the concrete cost of getting it wrong. A quantized YOLOv4-Tiny detector on a Raspberry Pi 5 cut inference time from 262 ms to 183 ms and power draw from 7.13 W to 4.00 W through INT8 post-training quantization alone, at a 3.5× smaller model size [2]. On true microcontroller-class hardware, a quantized MobileNetV2 detector integrated with a camera and Wi-Fi radio achieved 3.47–14.98 ms inference and 10.6–22.1 mJ energy per inference while holding accuracy at or above 0.85, demonstrating that battery-powered, real-time object

*Corresponding author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
AIoT '26, Tokyo, Japan
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/26/11
https://doi.org/XXXXXXX.XXXXXXX

detection at the extreme edge is achievable when quantization is engineered carefully [1]. Because these gains compound across fleets of devices rather than a single unit, even a two-fold improvement in per-device latency, or a few retained points of accuracy, has outsized significance for distributed, IoT-scale machine learning: it changes which detectors are viable at all on hardware costing a few dollars, without adding GPUs, NPUs, or cloud connectivity to the bill of materials.

Prior empirical work, however, has focused on architectures and hardware where the accuracy-preserving path is comparatively forgiving. Novac et al. [10] benchmark quantization and deployment across several microcontroller targets, but the networks under study use conventional convolutional stacks rather than the extreme regression heads found in modern one-to-one detectors. Boddu and Mukherjee [2] and Bhushan et al. [1] both show that INT8 post-training quantization of YOLO-family and MobileNet-family detectors preserves acceptable accuracy on ARM-based boards (Raspberry Pi, Cortex-M4), but neither targets a RISC-V SIMD accelerator, and neither model uses an NMS-free, RegMax=1 direct-regression head, where a single scalar carries each box coordinate with no distribution to absorb quantization noise. Closer to our application domain, TinyissimoYOLO [8] deploys a quantized YOLO-family detector across several microcontroller targets and reports sub-millijoule energy per inference; however, it targets SoCs with a dedicated CNN accelerator (MAX78000, GAP9) rather than a bare SIMD-only RISC-V core with no NPU, and like the works above it neither addresses an NMS-free, RegMax=1 head nor validates bit-exact parity between its quantization toolchain and the deployed firmware. Critically, none of these studies validates its quantization toolchain against the exact fixed-point arithmetic executed by the target silicon; each instead trusts a floating-point or generic integer simulation to predict on-device accuracy. This leaves an open research gap: no published deployment demonstrates an NMS-free YOLO26-class detector on a memory-bound, GPU-free RISC-V microcontroller such as the ESP32-P4, and no prior quantization study closes the gap between how a toolchain simulates fixed-point arithmetic and how the target hardware actually executes it. The research question this paper answers is therefore twofold: can a quantization-fragile, NMS-free detector be deployed correctly—with numerically verified, not merely estimated, accuracy—on a $5 RISC-V microcontroller; and can co-designing the model graph with the hardware's SIMD kernels recover the latency that quantization alone leaves on the table?

This paper addresses that gap with four objectives. First, we deliver the first documented deployment of YOLO26n on the ESP32-P4, resolving the quantization fragility of its depthwise, direct-regression head through mixed-precision INT16/INT8 quantization. Second, we introduce a general-purpose Neural Morphing Engine that restructures the model graph through label-free, quality-gated, block-wise distillation, producing operators that map onto efficient SIMD kernels. Third, we close the simulation gap directly by validating every intermediate value of the deployed model bit-exactly against the physical hardware, enabling full-dataset COCO evaluation without requiring thousands of on-device inference runs. Fourth, we release the resulting toolchain as an open-source, upstream contribution to Espressif's esp-dl SDK. The implications extend beyond this one detector: a validated methodology for shipping correct, fast, NMS-free perception on sub-$5, GPU-free, RISC-V microcontrollers is a necessary prerequisite for distributed machine learning across fleets of such devices, where per-unit cost and per-inference energy—not raw throughput—are typically the binding constraints. Realizing that potential, rather than merely enabling it in principle, requires demonstrating numeric consistency across multiple physical nodes, which this single-device study does not yet show; we return to this explicitly as future work in Section 4.

2 Methodology

2.1 Target Platform and Model

We target the ESP32-P4, a dual-core RISC-V microcontroller clocked at 400 MHz, equipped with a PIE SIMD coprocessor capable of 16 INT8 multiply-accumulate operations per cycle, 768 KB of on-chip SRAM, and 32 MB of external PSRAM, at a unit cost of approximately $5. The device has no GPU or NPU, which rules out the kernel libraries and memory assumptions that underpin most edge-AI deployments. We deploy YOLO26n, the 2.4M-parameter nano variant of the January-2026 YOLO26 family, which combines C3k2 backbone blocks, PSA attention, and an NMS-free detection head. This head uses RegMax = 1 direct regression, meaning each bounding-box coordinate is carried by a single scalar rather than a distribution over bins; consequently, the head's depthwise convolutions have very few weights per layer and are extremely sensitive to quantization noise, a property that made every naive quantization strategy we attempted fail outright.

2.2 Baseline Deployment and Mixed-Precision Quantization

We first established that INT8 quantization applied uniformly across the network destroys bounding-box regression, since RegMax = 1 regression offers no averaging across bins to absorb quantization error. We next tried INT16 convolutions with INT8 activations, which instead created a bottleneck at the INT8 Swish activation. A naive INT16 Swish implementation restored accuracy but added roughly 660 ms per layer, an unacceptable latency cost. We resolved this with an INT16 lookup-table (LUT) approximation of the Swish/Sigmoid activations, using linear interpolation between pivots, which reached 2,072 ms of inference latency at 0.365 mAP50-95—our deployable baseline.

Reproducing this LUT arithmetic in the Python quantization toolchain proved non-trivial: naive simulation dequantizes to float32 and evaluates the ideal activation function, while the firmware instead executes fixed-point interpolation with C-style truncation toward zero. We therefore built `esp_ppq_lut`, a custom extension to Espressif's ESP-PPQ toolkit that reproduces this arithmetic bit-for-bit in pure integer operations, branching explicitly on the sign of the numerator to emulate truncation rather than Python's floor-toward-negative-infinity division. This baseline pipeline—mixed-precision quantization plus the bit-exact LUT emulator—was accepted upstream into Espressif's official esp-dl repository as Pull Request #286, following review by four Espressif engineers over five weeks.

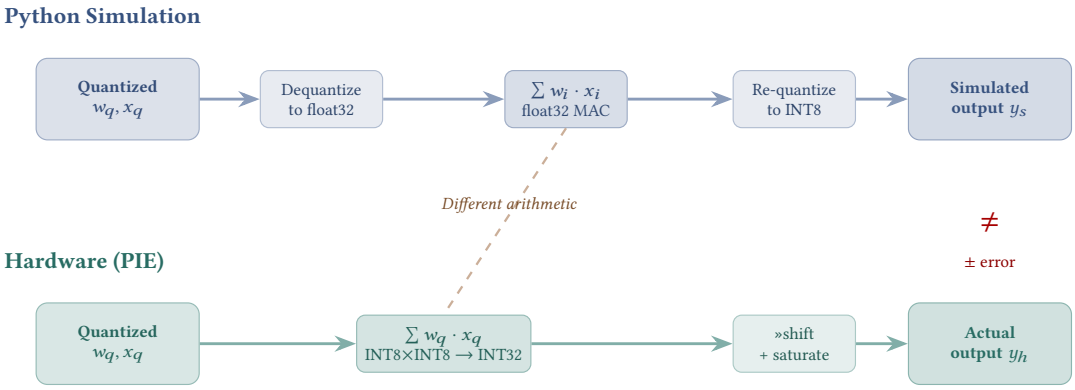


Figure 1: The simulation gap. The quantization toolchain (top) dequantizes to float32 and applies the ideal function, while the ESP32-P4 PIE (bottom) uses fixed-point arithmetic with truncation toward zero, a fidelity loss present in both the MAC accumulation and the activation lookup table. On the INT16 LUT primitive we emulate, the two paths disagree on 88% of output values.

2.3 Neural Morphing Engine (Model-Level Optimization)

Beyond quantization, we developed a Neural Morphing Engine (NME) to restructure the computation graph itself into forms that admit efficient SIMD execution. The NME operates block by block through a five-phase pipeline: (1) select candidate operations for replacement; (2) splice in a shape-compatible replacement subgraph; (3) train the replacement via label-free knowledge distillation [5] against a frozen teacher, using 512 unlabeled images; (4) apply a dual-threshold quality gate requiring cosine similarity $\cos_b \geq 0.99$ and an output magnitude ratio ρ_b within $[0.985, 1.015]$; and (5) log the outcome and proceed, with automatic rollback to the original operation on gate failure. This block-wise, teacher-anchored reconstruction follows the logic established by BRECQ for post-training quantization [7], which we generalize here from a purely numerical (precision-recovery) transformation to a structural one.

We evaluated three transformation types under this engine. T1 replaces the SiLU activation with HardSiLU8, a single-bit-shift, integer-friendly piecewise-linear gate in the spirit of MobileNetV3's HardSwish [6] but with a learned per-layer scale trained via a straight-through estimator; T1 achieved 66 candidate replacements, 45 accepted (68%), and cost only −0.005 mAP, recovering 98.6% of baseline accuracy. T2 decomposes 3×3 convolutions into depthwise-plus-pointwise pairs; T3 prunes the 16 least-important input channels of 1×1 convolutions. T2 and T3 were evaluated but not deployed, since rejection rates rose sharply with structural invasiveness (32% for T1, 63% for T2, 87% for T3), confirming that only T1 met our accuracy bar while T2 and T3 demonstrate the engine's generality across transformation types, as Figure 2 shows.

2.4 Hardware Kernel Co-Design (Hardware-Level Optimization)

Model-level transformation alone does not translate into latency reduction unless the resulting operators are matched to hardware-native kernels. We therefore implemented four custom PIE SIMD kernels. TransposePIE accelerates the data-movement transposes

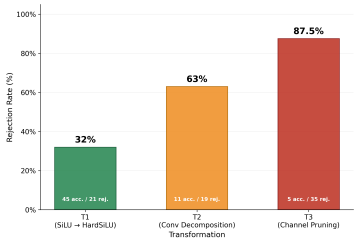


Figure 2: Rejection rate rises with structural invasiveness across the three Neural Morphing Engine transformations: 32% for T1 (SiLU → HardSiLU), 63% for T2 (convolution decomposition), and 87.5% for T3 (channel pruning).

required by PSA attention. NN-LUTINT16 replaces linear interpolation with nearest-neighbour indexing for INT16 activation lookups, exploiting the PIE's banker's-rounding multiply instruction. HardSiluPie8 derives an exact integer execution path for the T1-morphed activation. TiledConvBlock fuses convolution and activation into cache-resident tiles sized to fit the 768 KB L2 cache, avoiding round-trips to the comparatively slow 32 MB PSRAM. Applied incrementally, these kernels reduced inference latency from the 2,072 ms baseline to 1,088 ms: TransposePIE (1,769 ms), NN-LUTINT16 (1,613 ms), HardSiluPie8 (1,482 ms), TiledConvBlock spatial tiling (1,156 ms), TiledConvBlock for 1×1 convolutions (1,103 ms), and a final vector-hoisting pass (1,088 ms). TiledConvBlock is the single largest contributor, reflecting our central finding that the ESP32-P4 is memory-bound rather than compute-bound: the `model.7/conv` layer alone drops from 305 ms to 23 ms (13.3×) once its output tile is kept resident in cache.

We formalize this memory-bound claim with a standard roofline analysis [11] rather than leaving it as narrative. The PIE's peak compute throughput is 16 MACs/cycle × 400 MHz = 6.4 GMAC/s; the PSRAM bandwidth is 184 MB/s; the resulting ridge point is $6.4\times10^9/184\times10^6 \approx 34.8$ MACs/byte, matching the ≈ 35 MACs/byte figure in Section 1. Any kernel whose arithmetic intensity falls

below this ridge is memory-bound regardless of how few MACs it performs—precisely the regime the un-fused `Conv+Act` baseline operates in, since it round-trips the full activation tensor through PSRAM between the two ops. TiledConvBlock raises arithmetic intensity by keeping the intermediate tile in the 256 KB L2 cache instead of PSRAM, which is the mechanism—not a reduction in MACs—behind the 13.3× improvement on `model.7/conv`. We did not instrument per-layer attained GOPS against this roofline in the present study; doing so for each of the six kernel stages in Table 1, and plotting their operating points against the ridge, is an inexpensive follow-up we leave for the camera-ready version.

2.5 Validation Methodology

To validate the deployment, we adopted a four-test firmware protocol. Test 0 checks input-preprocessing parity by feeding raw, uncompressed RGB data through pipelines reproduced identically in Python and firmware. Test 1 compares full-model, on-device output against the Python digital twin across all 451,584 intermediate output values—the core bit-exactness claim. Test 2 is a control that bypasses the digital twin entirely, running the on-device output against naive floating-point math, to confirm that Test 1's parity is not an artifact of a trivial test harness. For the optimized (post-morphing, post-kernel) model, we instead validate at the detection level: all five detections on a reference image (class, confidence, and bounding-box coordinates) are compared byte-for-byte between the x86 Python simulation and the RISC-V firmware, shown in Figure 3. This bit-exact validation, being exhaustive over every intermediate value rather than sampled, is what allows us to run full 5,000-image COCO val2017 evaluation on x86 in seconds rather than requiring the roughly 91 minutes that 5,000 on-device inferences at the measured 1,088 ms median would otherwise demand.

2.6 Reproducibility and Artifact Availability

The baseline deployment (PR #286) is merged into, and the optimized kernel set (PR #310) is submitted to, Espressif's public `esp-dl` repository; `esp_ppq_lut` and `esp32-p4-jit` are released separately under the MIT license. We will accompany the camera-ready version with the exact COCO train2017 image lists used for calibration (2,560 images) and NME distillation (512 images), together with a pinned environment specification (ESP-IDF v5.3, ESP-DL 3.x, and the Python/PyTorch versions used for `esp_ppq_lut`), so that the reported 0.365 → 0.361 mAP pipeline is independently re-runnable end to end.

3 Results and Discussion

3.1 Baseline and Quantization Outcome

Our mixed-precision quantization pipeline delivered the first documented deployment of YOLO26n on the ESP32-P4: 2,072 ms inference at 0.365 mAP50-95, already 1.33× faster than the official YOLOv11n deployment on the same platform. This outcome was not the first attempt but the fourth: uniform INT8 quantization destroyed box regression outright, INT16-convolution-with-INT8-activation created a bottleneck at the Swish activation, and a naive INT16 Swish restored accuracy at the cost of +660 ms per layer. Only the combination of mixed-precision quantization and a custom bit-exact INT16 LUT emulator (`esp_ppq_lut`) resolved the



Figure 3: On-device ESP-DL emulated inference on `bus.jpg` (5 detections). By Test 1, class, confidence, and bounding-box coordinates are bit-identical to the x86 Python digital twin.

fragility of the depthwise, direct-regression head while remaining fast enough to deploy. This baseline was accepted into Espressif's official esp-dl SDK as PR #286, providing external, vendor-side validation of the approach beyond our own benchmarks.

3.2 Model-Level Optimization: Neural Morphing Engine

The Neural Morphing Engine's three evaluated transformations show a consistent pattern: rejection rate rises sharply with structural invasiveness. T1 (SiLU → HardSiLU8, a purely element-wise activation swap) was accepted in 45 of 66 candidate blocks (68%) at a cost of only −0.005 mAP, recovering 98.6% of baseline accuracy, and was the only transformation deployed. T2 (convolution decomposition, which alters weight structure) was accepted in 11 of 30 candidates (37%); T3 (channel pruning, which removes information entirely) was accepted in only 5 of 40 (13%), with a −0.025 mAP cost. This 32% → 63% → 87% escalation in rejection rate (Figure 2) is itself informative: it demonstrates that the engine's quality gate (cosine similarity and magnitude-ratio thresholds with automatic rollback) correctly discriminates between transformations that preserve function and those that do not, and that T2 and T3, while not deployed, validate the engine's generality across fundamentally different transformation classes rather than being failures of the method.

3.3 Hardware-Level Optimization

Table 1 shows inference dropping from 2,072 ms to 1,088 ms across four kernel families. TiledConvBlock alone accounts for 326 ms of the total 984 ms saved—the largest single contribution—because it

Table 1: Incremental latency reduction from hardware kernel co-design, relative to the 2,072 ms mixed-precision baseline.

Kernel stage	Latency (ms)	Reduction
Mixed-precision baseline	2,072	—
TransposePIE	1,769	14.6%
NN-LUTINT16	1,613	22.2%
HardSiluPie8	1,482	28.5%
TiledConvBlock (spatial tiling)	1,156	44.2%
TiledConvBlock (1×1 conv)	1,103	46.8%
Vector hoisting	1,088	47.5%

fuses convolution and activation into tiles sized to remain resident in the 768 KB L2 cache, eliminating round-trips to the far slower 32 MB PSRAM. The clearest illustration is layer `model.7/conv`, which drops from 305 ms to 23 ms, a 13.3× improvement, purely from this cache-aware rewrite with no change to the arithmetic performed.

3.4 Overall Results

Combining both optimization levels, the final deployed model runs at 1,088 ms—a 1.90× (47.5%) speedup over the 2,072 ms mixed-precision baseline—at 0.361 mAP50-95, only −3.7% relative to the FP32 reference of 0.375. Measured against the platform's prior deployment, this is a 2.54× speedup over YOLOv11n at comparable accuracy (0.361 vs. 0.360 mAP). Validation confirms 100% bit-exact parity between the Python digital twin and on-device output: 0 errors across 451,584 output values for the baseline model, and 5-of-5 identical detections (class, confidence, and box coordinates) for the optimized model (Figure 3), compared to 88% of values diverging when the emulator is bypassed entirely.

3.5 Energy: An Order-of-Magnitude Estimate

We report latency only in the measurements above; per-inference energy, a first-class metric for battery-operated IoT devices, requires an inline current-sense measurement (e.g., a shunt monitor on the supply rail) that we have not yet performed. In its absence we give a coarse estimate rather than none. Espressif's hardware design guidelines require a supply capable of at least 380–430 mA at 3.3 V (flash and PSRAM included) [3], i.e. roughly 1.25–1.42 W. Multiplied by the 1,088 ms median latency, this gives on the order of 1.4–1.5 J per inference. This number is *not* a measurement and not a strict bound in either direction: it is a supply-provisioning figure rather than a measured active-mode draw, and it ignores the incremental draw of the PIE under SIMD load. It should be read only as an order-of-magnitude indicator. It is also not comparable to accelerator-equipped platforms that report sub-millijoule inference energy [8], which rely on a dedicated low-power CNN engine the ESP32-P4 lacks. Direct current-sense measurement is the immediate next step.

3.6 Discussion

These results support three interpretations. First, the memory-bound finding—that the largest latency gain (TiledConvBlock, 326 ms) comes from a cache-aware data-movement rewrite rather than any reduction in arithmetic operations—indicates that TinyML kernel design for RISC-V edge SoCs should prioritize the memory hierarchy over FLOP counting, a conclusion that runs counter to the parameter-count and FLOPs metrics still commonly used to compare edge models. Second, the value of bit-exact simulation is not merely a validation nicety: it is what makes full-dataset COCO evaluation computationally tractable at all, replacing an estimated 91 minutes of on-device inference with a seconds-long x86 run, and it is what allows any accuracy number reported in this line of work to be trusted rather than merely estimated. Third, the acceptance of both the baseline (PR #286) and the optimized kernel set (PR #310) into Espressif's official esp-dl library constitutes an external, independent form of validation beyond our own benchmarking—the maintainers' review explicitly praised the three fusion operators (TransposePIE, TiledConvBlock, NN-LUT) as the technical core of the contribution.

The evaluation is nonetheless scoped to one detector (YOLO26n), one dataset (COCO val2017), and one platform (ESP32-P4); the bit-exact twin is specific to this chip's INT16 LUT primitive, so porting the methodology to a different accelerator would require re-deriving its arithmetic model rather than reusing `esp_ppq_lut` directly. T2 and T3 were evaluated but not deployed due to compounding accuracy loss, and energy per inference was not measured directly (Section 3.5). A reasonable counterargument is that a 1.90× speedup demonstrated on a single microcontroller is a narrow result. We consider this rebutted by three facts: the Neural Morphing Engine is transformation-agnostic and was shown to generalize across three structurally distinct transformation types (T1–T3); the validation methodology (bit-exact parity, not sampled or estimated accuracy) is itself a reusable contribution independent of the specific model; and the toolchain (`esp_ppq_lut`, `esp32-p4-jit`) is released open-source under the MIT license and has already been merged into a widely used vendor SDK, giving the contribution a path to reuse well beyond this paper.

Four further gaps deserve explicit acknowledgment rather than silent omission. First, every quantization baseline in Section 2.2 is our own; we have not compared against an established post-training rounding method such as AdaRound [9] applied to the same INT16 layers without our bit-exact emulator. That ablation would isolate how much of the accuracy recovery comes from emulator fidelity versus the quantization algorithm, and we regard it as the most informative experiment missing here. Second, T2 and T3 were evaluated at a fixed 512-image distillation budget and fixed gate thresholds ($\cos_b \geq 0.99$, $\rho_b \in [0.985, 1.015]$). Whether they are fundamentally harder to recover or merely under-resourced relative to T1 is not established; a sweep over distillation-set size and training steps, together with a threshold-sensitivity sweep on T1, would settle it. Third, our cross-platform argument rests on the ESP32-S3 sharing the PIE ISA, but we have not executed that port; it is the cheapest direct test of the generality claim. Fourth, all accuracy results use COCO val2017 at 512×512; object-size distributions in other IoT settings (e.g., small-object-heavy aerial imagery) may shift the accuracy–latency trade-off, which we have not measured.

4 Conclusion

The problem this paper set out to solve was concrete: the NMS-free YOLO26n detector, whose depthwise, direct-regression head is unusually sensitive to quantization noise, had no working deployment on the ESP32-P4, a $5, GPU-free RISC-V microcontroller. Four uniform quantization strategies were tried in sequence, and three of them failed for identifiable reasons—INT8 everywhere destroyed box regression, INT16-Conv-with-INT8-activation bottlenecked at the activation function, and naive INT16 Swish restored accuracy but at an unacceptable +660 ms per layer—before mixed-precision INT16/INT8 quantization paired with a custom bit-exact LUT emulator produced a working, deployable baseline at 2,072 ms and 0.365 mAP50-95.

From that baseline, a two-level co-design was applied: a Neural Morphing Engine restructured the model graph through label-free, quality-gated, block-wise distillation, while four custom PIE SIMD kernels were built to exploit the resulting structure and the chip's memory hierarchy. The evidence for this approach is direct and numerically exhaustive rather than sampled: inference latency fell from 2,072 ms to 1,088 ms (a 1.90× speedup), yielding 0.361 mAP50-95 (−3.7% from the FP32 reference) and a 2.54× speedup over the prior YOLOv11n deployment at comparable accuracy; and bit-exact validation confirmed 0 errors across 451,584 output values on-device, against 88% divergence when the validating emulator was removed.

From this evidence, three conclusions follow directly, and no further than the evidence supports. Multi-level co-design is necessary, not merely helpful, on memory-bound microcontrollers: the single largest latency contributor (TiledConvBlock, 326 ms) came from a cache-aware rewrite rather than any change to arithmetic complexity. Bit-exact emulation is what makes hardware-faithful evaluation of a full benchmark dataset practical, rather than a validation formality. And the approach generalizes beyond one lucky transformation: the Neural Morphing Engine's three evaluated transformation types (T1–T3) show a consistent, interpretable relationship between structural invasiveness and rejection rate, and both the baseline and optimized deployments were independently accepted into Espressif's production esp-dl SDK.

These conclusions carry a direct implication for the AIoT community: cheaper, faster, on-device object detection on commodity, GPU-free hardware is a necessary prerequisite for distributed machine learning across fleets of low-cost IoT devices, where unit cost and per-inference energy—not raw throughput—are usually the binding constraint. Realizing that potential requires demonstrating numeric consistency across multiple physical nodes, which this single-device study does not show. The claims are correspondingly bounded: this work covers one detector, one dataset, and one platform; T2 and T3 were evaluated but not deployed, at a distillation budget and gate threshold we have not yet swept; energy is given only as an order-of-magnitude estimate (Section 3.5); and no comparison against an established post-training quantization baseline such as AdaRound has been run. Future work will therefore, in priority order: (1) port this methodology to the ESP32-S3, which shares the PIE instruction set, as the cheapest direct test of generality; (2) add direct current-sense energy measurement; (3) run the AdaRound ablation to separate the emulator's contribution from the quantization algorithm's; (4) sweep the quality-gate thresholds and the T2/T3 distillation budget; and (5) build an MLIR backend to generalize the morphing strategy across model families, while completing upstream review of the optimized kernel set (PR #310).

Author Contributions

Billal Boumedine (first author): conceptualization, methodology, software (C++/PIE assembly kernels), validation, formal analysis, investigation, data curation, writing—original draft, visualization, project administration; developed `esp_ppq_lut`, the Neural Morphing Engine, and all four PIE kernels (HardSiluPie8, TransposePIE, TiledConvBlock, NN-LUT); merged the baseline into Espressif's `esp-dl` (PR #286). **Amina Belkacem**: software, validation, investigation, writing—review & editing, visualization; contributed to the baseline deployment and the bit-exact validation protocol. **Naima Boukhiar** (corresponding author): resources, funding acquisition, supervision, logistical support, writing—review & editing. All authors reviewed and approved the final manuscript.

References

[1] Ch Madhu Bhushan, Priya Koppuravuri, Nomitha Prasanthi, Firoj Gazi, Mohammad Muzakkir Hussain, Mohammad Abdussami, Aguru Aswani Devi, and Jamilurahman Faizi. 2025. Deploying TinyML for Energy-Efficient Object Detection and Communication in Low-Power Edge AI Systems. *Scientific Reports* 15 (2025), 44299. doi:10.1038/s41598-025-27818-9

[2] Sindhu Boddu and Arindam Mukherjee. 2025. Efficient Edge Deployment of Quantized YOLOv4-Tiny for Aerial Emergency Object Detection on Raspberry Pi 5. *arXiv preprint arXiv:2506.09300* (2025).

[3] Espressif Systems. 2026. ESP32-P4 Hardware Design Guidelines. https://docs.espressif.com/projects/esp-hardware-design-guidelines/en/latest/esp32p4/index.html.

[4] Jose-Carlos Gamazo-Real, Raúl Torres Fernández, and Adrián Murillo Armas. 2024. Comparison of Edge Computing Methods in Internet of Things Architectures for Efficient Estimation of Indoor Environmental Parameters with Machine Learning. *Engineering Applications of Artificial Intelligence* (2024). arXiv:2403.08810

[5] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. 2015. Distilling the Knowledge in a Neural Network. *arXiv preprint arXiv:1503.02531* (2015).

[6] Andrew Howard, Mark Sandler, Grace Chu, Liang-Chieh Chen, Bo Chen, Mingxing Tan, Weijun Wang, Yukun Zhu, Ruoming Pang, Vijay Vasudevan, Quoc V. Le, and Hartwig Adam. 2019. Searching for MobileNetV3. In *IEEE/CVF International Conference on Computer Vision (ICCV)*. 1314–1324.

[7] Yuhang Li, Ruihao Gong, Xu Tan, Yang Yang, Peng Hu, Qi Zhang, Fengwei Yu, Wei Wang, and Shi Gu. 2021. BRECQ: Pushing the Limit of Post-Training Quantization by Block Reconstruction. In *International Conference on Learning Representations (ICLR)*.

[8] Julian Moosmann, Marco Giordano, Christian Vogt, and Michele Magno. 2023. TinyissimoYOLO: A Quantized, Low-Memory Footprint, TinyML Object Detection Network for Low Power Microcontrollers. In *IEEE 5th International Conference on Artificial Intelligence Circuits and Systems (AICAS)*. doi:10.1109/AICAS57966.2023.10168657

[9] Markus Nagel, Rana Ali Amjad, Mart Van Baalen, Christos Louizos, and Tijmen Blankevoort. 2020. Up or Down? Adaptive Rounding for Post-Training Quantization. In *International Conference on Machine Learning (ICML) (Proceedings of Machine Learning Research, Vol. 119)*. 7197–7206.

[10] Pierre-Emmanuel Novac, Ghouthi Boukli Hacene, Alain Pegatoquet, Benoît Miramond, and Vincent Gripon. 2021. Quantization and Deployment of Deep Neural Networks on Microcontrollers. *Sensors* 21, 9 (2021), 2984. doi:10.3390/s21092984

[11] Samuel Williams, Andrew Waterman, and David Patterson. 2009. Roofline: An Insightful Visual Performance Model for Multicore Architectures. *Commun. ACM* 52, 4 (2009), 65–76. doi:10.1145/1498765.1498785